THE ODIN PROJECT · INTERMEDIATE HTML AND CSS

Intermediate HTML and CSS



Aykhan Dadashov · Web Technologies · 14.10.2026

01

Intermediate HTML and CSS: what I covered

Intermediate HTML

- Introduction
- SVG
- Tables

Intermediate CSS

- Default Styles
- CSS Units
- More Text Styles
- More CSS Properties
- Advanced Selectors
- Positioning
- CSS Functions
- Custom Properties
- Browser Compatibility
- Frameworks and Preprocessors

Forms

- Form Basics
- Form Validation
- **Project: Sign-up Form**

Grid

- Introduction to Grid
- Creating a Grid
- Positioning Grid Elements
- Advanced Grid Properties
- Using Flexbox and Grid
- **Project: Admin Dashboard**

02 SVG and tables

SVG – vector images written in XML, they stay sharp at any size

viewBox – the coordinate system of the drawing, so it can scale

Shapes – <circle>, <rect>, <path>; colors with fill and stroke

Tables – only for tabular data, not for page layout

Table tags – <table>, <tr>, <th>, <td>, <thead>, <tbody>, colspan

intermediate.html

```
<!-- a simple SVG icon -->
<svg viewBox="0 0 100 100" width="80">
  <circle cx="50" cy="50" r="40"
          fill="gold" stroke="black" />
</svg>

<!-- a simple table -->
<table>
  <thead>
    <tr><th>Subject</th><th>Mark</th></tr>
  </thead>
  <tbody>
    <tr><td>Web Tech</td><td>9</td></tr>
    <tr><td>Programming</td><td>8</td></tr>
  </tbody>
</table>
```

03 Default styles, units and text

Default styles

Every browser has its own default styles (margins on body, sizes of headings). A CSS reset or normalize makes the start the same in every browser.

CSS units

Absolute: px. Relative: em (parent font size), rem (root font size), % and vh / vw (viewport). I use rem for font sizes and px for borders.

More text styles

font-family with fallbacks, Google Fonts or @font-face, font-weight, letter-spacing, line-height, text-transform, text-shadow and text-overflow: ellipsis.

More CSS properties

background shorthand, border-radius, box-shadow, overflow, opacity. Small properties that make a page look finished.

04

Advanced selectors

Child and sibling – > (direct child), + (next sibling), ~ (all next siblings)

Pseudo-classes – a state: :hover, :focus, :first-child, :nth-child()

Pseudo-elements – a part: ::before, ::after, ::first-letter, ::marker

Attribute selectors – [type="email"], [href^="https"]

Result – I can style elements without adding many extra classes

selectors.css

```
/* only direct children */
.menu > li { display: inline-block; }

/* every second row of a table */
tr:nth-child(even) { background: #f2f2f2; }

/* input that has focus */
input:focus { outline: 2px solid gold; }

/* add an icon before a link */
a[href^="https"]::before {
  content: "↗ ";
}

/* paragraph right after a heading */
h2 + p { font-size: 1.2rem; }
```

05

Positioning

static – the default, normal flow of the page

relative – moves from its normal place, keeps its space

absolute – taken out of the flow, placed inside the closest positioned parent

fixed – stays in the same place on the screen when scrolling

sticky – normal until you scroll to it, then it sticks

position.css

```
.card {
  position: relative;  /* parent */
}

.badge {
  position: absolute;  /* inside .card */
  top: 10px;
  right: 10px;
}

header {
  position: sticky;    /* my profile nav */
  top: 0;
}
```

06

CSS functions and custom properties

calc() – mix units, e.g. 100% minus the header height

min(), max(), clamp() – sizes that adapt to the screen

Custom properties – variables like --main-color, used with var()

Scope – defined on :root they work everywhere, on an element only inside it

Themes – change a few variables to make a dark theme

functions.css

```
:root {
  --ink: #1b1b22;
  --accent: #f7df1e;
}

h1 {
  /* min 40px, max 68px, grows with screen */
  font-size: clamp(40px, 7vw, 68px);
  color: var(--ink);
}

main {
  height: calc(100vh - 70px);
}

.button {
  background: var(--accent, gold);
}
```

07

Browsers, frameworks, preprocessors

Browser compatibility

Not every browser supports every new feature. I check caniuse.com before I use something new, and test in Chrome, Firefox and Safari.

Frameworks

Bootstrap and Tailwind give ready classes and components, so you write less CSS. But first you need to understand normal CSS.

Preprocessors

Sass, Less and PostCSS add features like nesting, mixins and loops, and compile to normal CSS. Today CSS itself has variables and nesting.

My choice

For these projects I used plain CSS without frameworks, to practice the basics that the course teaches.

08 Forms and form validation

<form> – action and method say where the data is sent

<label for> – connects text with an input, better for accessibility

Input types – email, tel, password, number, date, checkbox, radio

Validation – required, minlength, maxlength, min, max, pattern

Styling – :valid, :invalid and :user-invalid show errors

form.html

```
<form action="/signup" method="post">
  <label for="email">Email</label>
  <input type="email" id="email"
         name="email" required>

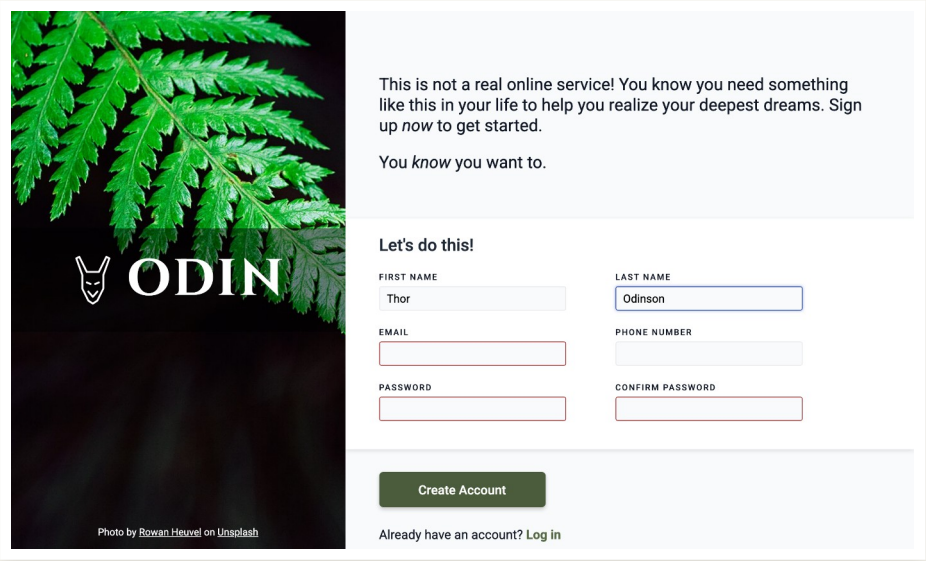
  <label for="password">Password</label>
  <input type="password" id="password"
         name="password" minlength="8"
         required>

  <button>Create Account</button>
</form>

/* red border only after the user typed */
input:user-invalid { border-color: red; }
```

09

Project: Sign-up Form



- Two columns made with Flexbox
- Image side with logo and photo credit
- Labels, input types and required fields
- Red border only after the user interacts
- Small script checks that passwords match

Live ayxan1110.github.io/odin-sign-up-form/

Code github.com/Ayxan1110/odin-sign-up-form

10

CSS Grid

Grid container – display: grid turns children into grid items

Tracks – grid-template-columns / rows, the fr unit, repeat()

Gap – space between rows and columns

Placing items – grid-column: 1 / 3, grid-row, grid-template-areas

Responsive – repeat(auto-fit, minmax(240px, 1fr)) without media queries

grid.css

```
.dashboard {
  display: grid;
  grid-template-columns: 250px 1fr;
  grid-template-rows: auto 1fr;
  min-height: 100vh;
}

.sidebar {
  grid-row: 1 / 3;   /* full height */
}

.cards {
  display: grid;
  grid-template-columns:
    repeat(auto-fit, minmax(240px, 1fr));
  gap: 20px;
}
```

11

Flexbox or Grid?

Flexbox: one direction

Good for a row or a column of items: a navigation bar, buttons in a row, centering one thing. The content decides the size.

Grid: two directions

Good for the whole page layout or a gallery of cards, where rows and columns must line up. The layout decides the size.

Together

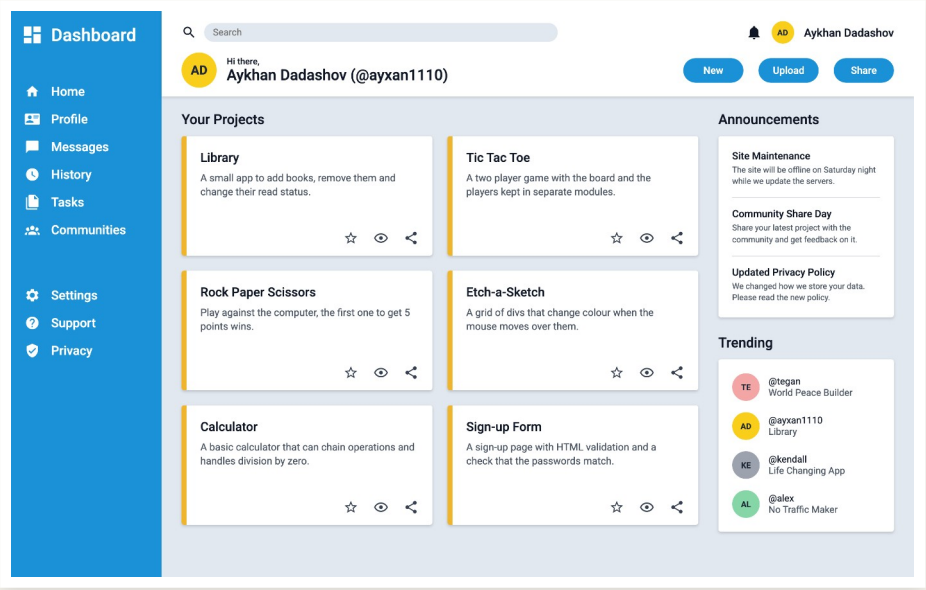
In the Admin Dashboard the page and the cards use Grid, and the header, buttons and items inside a card use Flexbox.

Advanced grid

auto-fill vs auto-fit, minmax(), implicit tracks with grid-auto-rows, and naming lines and areas.

12

Project: Admin Dashboard



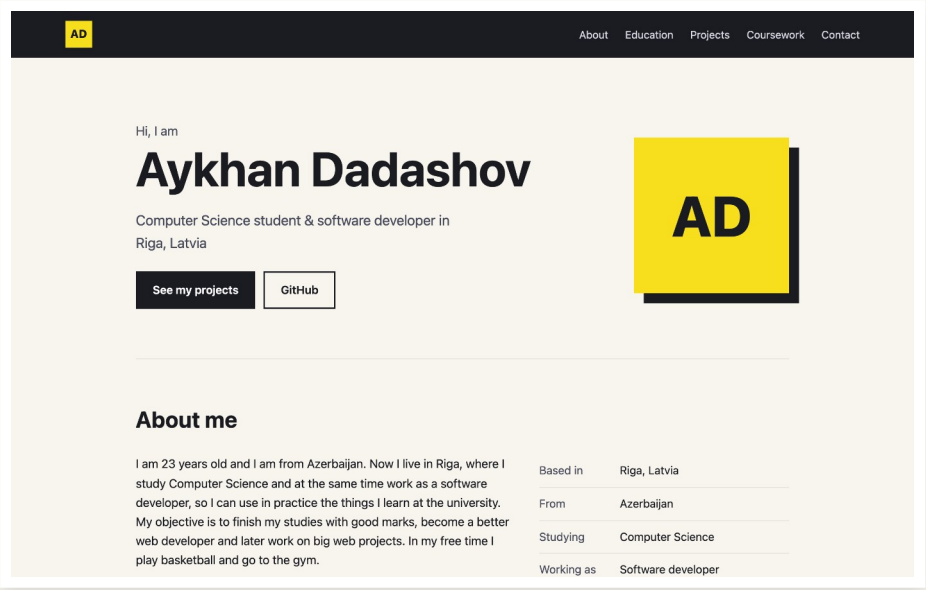
- Page layout made with CSS Grid
- Sidebar with icons (inline SVG)
- Header with search, avatar and buttons
- Project cards in a responsive grid
- Announcements and Trending in the right column

Live ayxan1110.github.io/odin-admin-dashboard/

Code github.com/Ayxan1110/odin-admin-dashboard

13

My profile page on GitHub Pages



- Hosted for free with GitHub Pages
- About me, education and skills
- Cards linking to all my projects
- Table with all Web Technologies tasks
- Sticky nav, CSS variables, clamp(), Grid

Live ayxan1110.github.io/

Code github.com/Ayxan1110/Ayxan1110.github.io

14 Projects on GitHub

Sign-up Form	ayxan1110.github.io/odin-sign-up-form/	github.com/Ayxan1110/odin-sign-up-form
Admin Dashboard	ayxan1110.github.io/odin-admin-dashboard/	github.com/Ayxan1110/odin-admin-dashboard
Profile page	ayxan1110.github.io/	github.com/Ayxan1110/Ayxan1110.github.io

Thank you!